

Package ‘BOMcatchr’

August 30, 2026

Type Package

Title Climate data anywhere in Australia and any time-step

Version 0.2.10

Maintainer Tim Peterson <tim.peterson@monash.edu>

Description NetCDF files of the Bureau of Meteorology Australian Water Availability Project daily national climate grids are built and used for the efficient extraction of point and catchment area weighted precipitation, minimum temperature, maximum temperature, vapour pressure, solar radiation and various measures of evapotranspiration. For details on the source climate data see <<http://www.bom.gov.au/jsp/awap/>>. Please cite <<https://doi.org/10.1002/hyp.13637>>.

Depends R (>= 4.0)

Imports archive, Evapotranspiration (>= 1.14), RNetCDF, utils, sf, zoo, methods, xts, stats, curl, RCurl, elevatr, progress, terra (>= 1.9), bigmemory

Suggests knitr, rmarkdown, testthat (>= 3.0.0),

VignetteBuilder knitr

BugReports <https://github.com/peterson-tim-j/BOMcatchr/issues>

URL <https://github.com/peterson-tim-j/BOMcatchr>,
<https://peterson-tim-j.github.io/BOMcatchr/>

License GPL-3

Encoding UTF-8

LazyData true

Config/testthat/edition 3

Config/roxygen2/version 8.1.0

Author Tim Peterson [aut, cre] (<<https://orcid.org/0000-0002-1885-0826>>),
Conrad Wasko [aut] (<<https://orcid.org/0000-0002-9166-8289>>)

R topics documented:

extract_data	2
extract_layer	7
extract_locations	8
extract_nstations	10
grid_ages	11
grid_build	12
grid_sources	14
grid_summary	15
weather_gauge_loc	16
weather_gauge_obs	17
Index	18

extract_data	<i>Extract data over catchment area and duration .</i>
--------------	--

Description

extract_data extracts the gridded climate data for each point or polygon using user-defined metrics and time-steps.

Usage

```
extract_data(  
  ncdfFilename = file.path(getwd(), "BOMcatchr_data.nc"),  
  extractFrom = as.Date("1900-01-01", "%Y-%m-%d"),  
  extractTo = as.Date(Sys.Date(), "%Y-%m-%d"),  
  vars = "",  
  locations = NA,  
  temporal.timestep = "daily",  
  temporal.fn.outer = NA,  
  temporal.fn.inner = "mean",  
  spatial.fn = "var",  
  interp.method = "",  
  missing.method = c("5", "linear", "mean"),  
  ET.function = "ET.MortonCRAE",  
  ET.DEM.res = 10,  
  ET.Mortons.est = "wet areal ET",  
  ET.Turc.humid = F,  
  ET.timestep = "monthly",  
  ET.missing_method = "DoY average",  
  ET.abnormal_method = "DoY average",  
  ET.constants = list()  
)
```

Arguments

<code>ncdfFilename</code>	is a full file name (as string) to the netCDF file.
<code>extractFrom</code>	is a date string specifying the start date for data extraction. The default is "1900-1-1".
<code>extractTo</code>	is a date string specifying the end date for the data extraction. The default is today's date as YYYY-MM-DD.
<code>vars</code>	is a vector of variables names to extract. The available variables are: daily maximum temperature, minimum temperature, daily precipitation, daily precipitation root-mean-square-error (RMSE), monthly precipitation, daily 9am and 3pm vapour pressure grids and daily solar radiation and evapotranspiration. The input vector for these options are <code>c('tmax', 'tmin', 'precip', 'precip.RMSE', 'precip.monthly', 'vprp_9am', 'vprp_3pm', 'solarrad', 'et')</code> . Importantly, the input <code>et</code> is calculated from the available gridded data (see ET. inputs below). To calculate the ET, all of the required inputs for the calculation ET must also be extracted (i.e. the input for such would generally be <code>c('tmax', 'tmin', 'precip', 'vprp_3pm', 'solarrad', 'et')</code>). Any or all of the defaults are available. The default <code>' '</code> and this will result in all of the variables in the netCDF file and provided by <code>rownames(BOMcatchr::grid_summary(ncdfFilename))</code> .
<code>locations</code>	is either an output <code>terra::vect</code> object from extract_locations , the full file name to an ESRI shape file of points or polygons (latter assumed to be catchment boundaries) or a shape file already imported using <code>readShapeSpatial()</code> . Either way the shape file must be in long/lat (i.e. not projected), use the EPSG:4283 datum (i.e. GDA94), and the first column must be a unique ID.
<code>temporal.timestep</code>	character string for the time step of the output data. The options are daily, weekly, monthly, quarterly, annual or a user-defined index for, say, water-years (see <code>xts::period.apply</code>). The default is daily.
<code>temporal.fn.outer</code>	character string or function object for complex aggregation of the raw daily or monthly source data to <code>temporal.timestep</code> . The function is applied after the aggregation of the raw data using the <code>temporal.fn.inner</code> function. This feature allows calculation of, say, the anomaly in the seasonal mean precipitation. The options are <code>'annom'</code> , <code>'cumannom'</code> or a user-defined function. The default is NA, resulting in no such calculations.
<code>temporal.fn.inner</code>	character string or function object for the aggregation of the raw daily or monthly source data and before more complex calculations defined by the <code>temporal.fn.outer</code> function. The options are <code>'sum'</code> , <code>'mean'</code> , <code>'min'</code> , <code>'max'</code> , <code>'median'</code> , <code>'sd'</code> , <code>'var'</code> , <code>'IQR'</code> or a user defined function requiring input of only a vector of values. The default is mean.
<code>spatial.fn</code>	character string for the function name applied to estimate the daily spatial spread in each variable. If NA or <code>''</code> and <code>locations</code> is a polygon, then the spatial data is returned. The default is <code>var</code> .
<code>interp.method</code>	character string defining the method for interpolating the gridded data (see <code>terra::extract</code>). The options are: <code>'simple'</code> , <code>'bilinear'</code> and <code>' '</code> . The default is <code>' '</code> . This

will set the interpolation to 'simple' when locations is a polygon(s) and to 'bilinear' when locations are points.

missing.method	three character vector for the settings to fill gaps in the source data. The three inputs control the following. # 1) the infilling of small holes in the source grids using focal(), which takes the mean of the non-NA surrounding grid cells. The user input controls the maximum hole size filled, in units of number of grid cells. Where the hole is greater than the input, a gap within the hole will remain. The default maximum hole size infilled is 5x5 grid cells. 2) Gaps that remain after the hole infilling, or time steps with no observations, are interpolated over time. Only gaps with observations prior to the gap are interpolated. The interpolation method is user-defined and includes 'constant', 'linear', 'fmm', 'periodic', 'natural', 'monoH.FC' and 'hyman'. The default is 'linear'. See approx and splinefun for details. 3) Gaps that remain (often due to the extraction date being prior to the start of the observation record of a variable) are estimated from, say, the mean for each day of the year. Specifically, the extracted observed data is allocated to each calendar day. If, say, there are ten years of daily data then each day of the year will have ten observations. All gaps of the same corresponding calendar day will then be assigned a value from a user-defined function from these observations (NB: when only one observed value exists for the day, then the observed value is returned). The default function is mean. Other standard functions (e.g. median) or user defined functions can be used. The default for this input is c('5', 'linear', 'mean'). All gap filling method can be turned off with the input c('0', '', ''), which is useful to identify the interpolated data points.
ET.function	character string for the evapotranspiration function to be used. The methods that can be derived from the gridded data are are ET.Abtew , ET.HargreavesSamani , ET.JensenHaise , ET.Makkink , ET.McGuinnessBordne , ET.MortonCRAE , ET.MortonCRWE , ET.Turc . Default is ET.MortonCRAE i.e. the complementary relationship for areal evapotranspiration .
ET.DEM.res	is the zoom resolution for the land surface elevation and is required to calculate the ET. elevatr package is used to extract elevation (metres) from AWS Open Data Terrain Tiles. This input controls the zoom resolution. Higher values increase accuracy, but are significantly slower. See details. Default is 10.
ET.Mortons.est	character string for the type of Morton's ET estimate. For ET.MortonCRAE , the options are potential ET, wet areal ET or actual areal ET. For ET.MortonCRWE , the options are potential ET or shallow lake ET. The default is wet areal ET, which when <code>ET.function = 'ET.MortonCRAE'</code> it provides an estimate of the wet areal potential evapotranspiration.
ET.Turc.humid	logical variable for the Turc function using the humid adjustment. See ET.Turc . For now this is fixed at F.
ET.timestep	character string for the evapotranspiration time step. Options are daily, monthly, annual but the options are dependent upon the chosen ET.function. The default is 'monthly'.
ET.missing_method	character string for interpolation method for missing variables required for ET calculation. The options are 'monthly average', 'seasonal average', 'DoY average' and 'neighbouring average'. Default is 'DoY average' but when

	the extraction duration is less than two years, the default is 'neighbouring average'. See ReadInputs
ET.abnormal_method	character string for interpolation method for abnormal variables required for ET calculation (e.g. Tmin > Tmax). Options and defaults are as for ET.missing_method. See ReadInputs
ET.constants	list of constants from Evapotranspiration package required for ET calculations. To get the data use the command <code>data(constants)</code> . Default is <code>list()</code> .

Details

Daily data is extracted and can be aggregated to a weekly, monthly, quarterly, annual or a user-defined timestep using a user-defined function (e.g. `sum`, `mean`, `min`, `max` as defined by `temporal.fn.outer`). The temporally aggregated data at each grid cell is then used to derive the spatial mean or the spatial variance (or any other function as defined by `spatial.fn`).

The calculation of the spatially weighted estimates uses (i) the fraction of each grid cell within, say, the catchment polygon and (ii) are area of each grid cell, which changes with longitude, divided by the mean area. Specifically, the fractional area of each grid cell is multiplied by the scaled area.

The variance calculation (or user defined function) does not use the fractional weight of the grid cell and returns NA if there are <2 grid cells in the catchment boundary. Prior to the spatial aggregation, evapotranspiration (ET) can also be calculated; after which, say, the mean and variance of the evaporation can be calculated.

The data extraction will by default be undertaken from 1/1/1900 to yesterday, else the minimum date range from all of the variables to be extracted.

The ET can be calculated using one of eight methods at a user defined calculation time-step; that is the `ET.timestep` defines the time step at which the estimates are derived and differs from the output timestep as defined by `temporal.fn.outer`). When `ET.timestep` is monthly or annual then the ET estimate is linearly interpolated to a daily time step (using `zoo::na.spline()`) and then constrained to ≥ 0 . In calculating ET, the input data is pre-processed using `Evapotranspiration::ReadInputs()` such that missing days, missing entries and abnormal values are interpolated (by default) with the former two interpolated using the "DoY average", i.e. replacement with same day-of-the-year average. Additionally, when solar radiation is required for the ET function, data is only available from 1/1/1990. To derive ET values <1990, the average solar radiation for each day of the year from 1/1/1990 to "extractTo" is derived (i.e. 365 values) and then applied to each day prior to 1990. Importantly, in this situation the estimates of ET <1990 are dependent upon the end date extracted. Re-running the estimation of ET with a later `extractTo` data will change the estimates of ET prior to 1990.

Some measures of ET require land surface elevation. Here, elevation at the centre of each 0.05 degree grid cell is obtained using the `elevatr` package, which here uses data from the Amazon Web Service AWS Open Data Terrain Tiles. The data sources change with the user set `ET.DEM.res.zoom`. The options are 1 to 15. The default of 10 is reasonably computationally efficient and has a resolution of about 108 m, with is acceptable given the 0.05 degree resolution of the BOM source data grids equates to about 5 km x 5 km. For details see <https://github.com/tilezen/joerd/blob/master/docs/data-sources.md>

Also, when "locations" is points (not polygons), then the netCDF grids are interpolate using bilinear interpolation of the closest 4 grid cells.

Lastly, data is extracted for all time points and no temporal infilling is undertaken if the grid cells are blank.

Value

When locations are polygons and `spatial.fn` is not NA or "", then the returned variable is a list variable containing two data.frames. The first is the areal aggregated climate metrics named `temporal`. The second is the measure of spatial variability named `spatial`.

When locations are polygons and `spatial.fn` does equal NA or "", then the returned variable is a `terra::vect` object where the first column is the location/catchment IDs and the latter columns are the results for each variable at each time point as defined by `temporal.timestep`.

When locations are points, the returned variable is a data.frame containing daily climate data at each point.

See Also

[grid_build](#) for building the NetCDF files of daily climate data. [grid_ages](#) for summarising the dates the gridded data was created by the BoM and downloaded, and what needs updating. [extract_nstations](#) for extracting the number of weather stations used at each time point to create the grids. [extract_locations](#) to get spatial data of catchment boundaries, rivers, etc.

Examples

```
# The example shows how to extract and save data.
#-----

# Set dates for building netCDFs and extracting data.
# Note, to reduce runtime this is done only a fortnight (14 days).
startDate = as.Date("2000-01-01", "%Y-%m-%d")
endDate = as.Date("2000-01-14", "%Y-%m-%d")

# Set names for netCDF file.
ncdfFilename = tempfile(fileext='.nc')

# Build netCDF grids and over a defined time period.
# Only precip data is to be added to the netCDF files.
# This is because the URLs for the other variables are set to zero.

file.name = grid_build(ncdfFilename=ncdfFilename,
                      updateFrom=startDate,
                      updateTo=endDate,
                      vars = c('precip'))

# Load example catchment boundary.
catch <- extract_locations('catchment', c('407214'))

# Extract daily precip. data (not Tmin, Tmax, VPD, ET).
# Note, the input "locations" can also be a file to a ESRI shape file.
climateData = extract_data(ncdfFilename=file.name,
                          extractFrom=startDate,
                          extractTo=endDate,
```

```

        vars = c('precip'),
        locations=catch,
        temporal.timestep = 'daily')

# Extract the daily catchment average data.
climateDataAvg = climateData$temporal

# Extract the daily catchment variance data.
climateDataVar = climateData$spatial

# Remove temp. files
unlink(ncdfFilename)

```

extract_layer	<i>Extract netCDF layer of one variable at one date.</i>
---------------	--

Description

extract_layer extracts gridded climate data layer for one date and one variable. This is low level function is unlikely to be of use to a user.

Usage

```

extract_layer(
  ncdfFilename = NA,
  ncdf.cond = NA,
  extract.date = NA,
  var = NA,
  vars.summary = NA
)

```

Arguments

ncdfFilename	is a full file name (as string) to the netCDF file.
ncdf.cond	connection to the netcdf file. Default is NA. If not provided the connection is established.
extract.date	is a date string specifying the date for data extraction.
var	is a character string one one variable to extract. The options are c('tmax', 'tmin', 'precip', 'precip.monthly', 'vprp', 'solarrad', 'et').
vars.summary	netCDF summary data.frame returned by grid_summary(ncdfFilename). Default is NA, which results in grid_summary(ncdfFilename) being called internally.

Value

terra::vect object.

See Also

[grid_build](#) for building the NetCDF files of daily climate data.

extract_locations

Get Australian Hydrological Geospatial Fabric data

Description

Selected Australian Hydrological Geospatial Fabric (Geofabric) spatial items are downloaded, allowing an efficient workflow from location selection to data extraction.

Usage

```
extract_locations(
  type = NA,
  id = NA,
  url_base = "https://hosting.wsapi.cloud.bom.gov.au/arcgis/rest/services/"
)
```

Arguments

type	is a character for the type of geofabric item to download. The options are: • 'state': Australian state or territory, including islands. Geofabric ID = 0 • 'water_gauge': Surface water gauging station. Geofabric ID = 2 • 'river_simple': simplified line vectors of rivers. • 'waterbody_simple': simplified polygons of stationary area of surface water (natural or constructed) • 'river_region': single large scale hydrological drainage basins. Geofabric ID = 34 • 'drainage_division': large scale collection of connected hydrological basins. Geofabric ID = 35 • 'catchment': single (often small) hydrological drainage basin with a stream gauge at the outlet. Geofabric ID = 49 • 'aquifer_upper': middle groundwater aquifer boundary and type. Geofabric ID = 57 • 'aquifer_mid': lower groundwater aquifer boundary and type. Geofabric ID = 58 • 'aquifer_lower': lower groundwater aquifer boundary and type. Geofabric ID = 59
id	is a scalar or vector of identifiers for the type of item to be downloaded. If NA, then all items will be downloaded. Otherwise the input variable depends on the type. The options are: • 'state': character string for the state. The options are 'ACT', 'NSW', 'NT', 'QLD', 'SA', 'TAS', 'VIC', 'WA'. Note, column 1 is an integer ID for each state and not a character label.

	• 'water_gauge' : character string for the "stationno" ID. • 'river_simple' : character string for the name of the river "ahgfnetwork". Note, not all rivers are named. • 'waterbody_simple' : integer for the Geofabric "hydroid". • 'river_region' : integer for the Geofabric "hydroid". • 'drainage_division' : integer for the Geofabric "divnumber" ID. • 'catchment' : character string for the Geofabric "stationno" ID at the catchment outlet. • 'aquifer_upper' : integer for the Geofabric "hydroid". • 'aquifer_mid' : integer for the Geofabric "hydroid". • 'aquifer_lower' : integer for the Geofabric "hydroid".
url_base	character string for URL to the BOM geofabric spatial data. Provided to allow editing in case the URL changes. The default is https://hosting.wsapi.cloud.bom.gov.au/arcgis/rest/services

Details

The function downloads a selected set of BOM Australian Hydrological Geospatial Fabric spatial vector data (points, lines and polygons) that likely to be relevant to hydrological, hydrogeological and aquatic ecological studies. All of the data is sourced from <https://www.bom.gov.au/water/geofabric/>. The spatial data is transformed to a GDA94 datum (i.e. EPSG:4283) to be consistent with the gridded weather data.

To find the required ID for the desired data type, either download all data and filter for the required location (see example) or use the online mapping tool at <https://portal.wsapi.cloud.bom.gov.au/arcgis/home/webmap/viewer.html?useExisting=1&layers=35719064c4ea4ad79faa82f5c9c22068>

Note, when id=NA all of the spatial data of the given type is downloaded. For some variables (e.g. 'catchment') there is a significant amount of data and the request may fail. If this occurs, try downloading the data in smaller sets.

Value

terra::SpatVector

See Also

[extract_data](#) for extracting climate data.

Examples

```
# Get state boundary, Loddon River drainage basin,
# all stream lines, all stream gauges and two catchment boundaries
st <- extract_locations('state','VIC')
dr <- extract_locations('river_region',43637156)
riv <- extract_locations('river_simple',NA)
gauge <- extract_locations('water_gauge',NA)
catch <- extract_locations('catchment',c('407214','407220'))

# Filter streamlines and gauges to those within the Loddon River basin.
gauge <- gauge[dr]
```

```

riv <- riv[dr]

# Map drainage basin and catchments in state
terra::plot(st, border='grey')
terra::plot(dr, border='black', add = TRUE)
terra::plot(catch, col='red', add = TRUE)

# Map catchments, streamlines and gauges
terra::plot(dr, border='black')
terra::plot(catch, border='red', add = TRUE)
terra::plot(riv, col='blue', add = TRUE)
terra::plot(gauge, col='grey', add = TRUE)

```

extract_nstations	<i>Age of netCDF data and updates required.</i>
-------------------	---

Description

extract_nstations gets number of observation stations per variable per time step.

Usage

```
extract_nstations(ncfile, vars = "")
```

Arguments

ncfile	file name of the netCDF data file built by the package function grid_build .
vars	is a vector of variables names to extract.

Details

This function gets the number of observation weather stations used to produce each grid at each time step. [grid_sources](#)),

Value

data.frames, one for each variable in the netCDF file. Each data frame contains the following columns:

- Date: observation date,
- n_stations: number of stations to the grid for the timestep.

See Also

[grid_build](#)

grid_ages	<i>Age of netCDF data and updates required.</i>
-----------	---

Description

grid_ages age of the netCDF date and which time points likely have BoM updates.

Usage

```
grid_ages(
  ncfile,
  data.src = grid_sources(),
  silent = F,
  plot.sourcedate = F,
  today = Sys.Date()
)
```

Arguments

ncfile	file name of the netCDF data file built by the package function grid_build .
data.src	is a data.frame of the source grid information. Default is from grid_sources .
silent	is a logical for silencing console outputs. Default is F.
plot.sourcedate	is a logical for plotting the date that the data for each time point was downloaded. Default is F.
today	is today's date. Default is Sys.Date(). This input is only provided to demonstrate the function within the vignettes.

Details

This function compares the download date of data within an existing netCDF file and compares them against the BoM schedule of data updates (provided (see [grid_sources](#)),

Value

list of data.frames, one for each variable in the netCDF file. Each data frame contains the following columns:

- date: observation date,
- source.date: date grid was downloaded from the BoM.
- create.date: date grid was created by BoM.
- source.age : age of observation in days.
- $1 < \text{age} \leq 7$: logical if source.age is greater than the left threshold and less than or equal to right duration threshold.
- $\text{age} \leq 7$: logical if source.age is less than or equal to the maximum duration threshold.

See Also

[grid_sources](#) for update threshold data.

grid_build

Build a netCDF file of climate data.

Description

grid_build builds one netCDF file containing Australian climate data.

Usage

```
grid_build(
  ncdfFilename = file.path(getwd(), "BOMcatchr_data.nc"),
  updateFrom = as.Date("1900-01-01", "%Y-%m-%d"),
  updateTo = as.Date(Sys.Date() - 2, "%Y-%m-%d"),
  vars = "",
  keepFiles = FALSE,
  compressionLevel = 5,
  vars.sourceData = grid_sources(),
  chunksizes = NULL
)
```

Arguments

ncdfFilename	is a file path (as string) and name to the netCDF file. If only a file name is given, then the file is assumed to exist / be created in getwd(). The default file name and path is file.path(getwd(), 'BOMcatchr_data.nc').
updateFrom	is a date string specifying the start date for the gridded data. If ncdfFilename is specified and exist, then the netCDF grids will be updated with new data from updateFrom. To update the file from the end of the last day in the file set updateFrom=NA. The default is "1900-1-1".
updateTo	is a date string specifying the end date for the gridded data. If ncdfFilename is specified and exist, then the netCDF grids will be updated with new data to updateFrom. The default is two days ago as YYYY-MM-DD.
vars	is a vector of variables names to build or update. The available variables are: daily daily maximum temperature, minimum temperature, daily precipitation, daily precipitation root-mean-square-error (RMSE), monthly precipitation, daily 9am and 3pm vapour pressure grids and daily solar radiation The input vector for these options are c('tmax', 'tmin', 'precip', 'precip.RMSE', 'precip.monthly', 'vprp_9am', 'vprp_3pm', 'solarrad'). Any or all of the defaults are available. If vars='' and the netCDF does not exist, then the default is all available variables as provided by rownames(grid_sources()). However, if vars='' and the netCDF file does exist, then default is to use the variable names in the file.
keepFiles	is a logical scalar to keep the downloaded gridded data files. The default is FALSE.

compressionLevel	is the netCDF compression level between 1 (low) and 9 (high), and NA for no compression. Note, data extraction runtime may slightly increase with the level of compression. The default is 5.
vars.sourceData	is a data.frame of variable unit, time step and source URLs. This input is provided in-case the default URLs need to be changed. The default is <code>grid_sources()</code>
chunksizes	netCDF chunk size expressed as the number of elements along each dimension (see <code>RNetCDF::var.def.nc</code> for details). Here the default is NULL, which informs the NetCDF library to use a default chunking strategy intended to give reasonable performance for building and reading the grids. Alternatively, to further reduce the time required to extract long time-series at a few grid cells the input can be changed to, say, <code>c(20, 20, 365)</code> , where the first two elements are for the spatial coordinate chunks and the third is for time. Importantly, such prioritising of the time dimension dramatically increases the time required to build the netCDF grids. Finally, the <code>chunksizes</code> can only be set when a new variable is added to a file. Once the variable is created, <code>chunksizes</code> cannot be modified.

Details

`grid_build` creates one netCDF file of daily climate data.

One netCDF file is created that contains (by default) daily precipitation, daily precipitation RMSE, minimum daily temperature, maximum daily temperature, daily vapour pressure at 9am and 3pm, daily solar radiation data, daily precipitation RMSE and monthly precipitation. The default data spans from 1/1/1900 to yesterday and requires ~50GB of hard-drive space (using default compression).

To minimise the runtime in extracting data, the netCDF file should be stored locally and not on a network drive.

The climate data is sourced from the Bureau of Meteorology Australian Water Availability Project (<http://www.bom.gov.au/jsp/awap/>). For details see Jones et al. (2009).

The output from this function is required for all data extraction functions within this package and must be ran prior.

The function can be used to a build netCDF file from scratch or to update an existing netCDF file previously derived from this function..

Value

A list of data.frames, one for each time step size (i.e. daily or monthly), giving an error code for each time step and variable. The codes are as follows:

- 1: Successfully downloaded timestep and added it to the netCDF file.
- 0: No code recorded (default). This is likely because the timestep was prior to the start of the available BOM data.
- -1: Error downloading the grid data for the timestep.
- -2: Error importing the downloaded data into the netCDF file.

References

David A. Jones, William Wang and Robert Fawcett, (2009), High-quality spatial climate data-sets for Australia, Australian Meteorological and Oceanographic Journal, 58 , p233-248.

See Also

[grid_summary](#) for summarising the built data. [grid_ages](#) for summarising the dates the gridded data was created by the BoM and downloaded, and what needs updating. [extract_data](#) for extracting data at points and polygons. [extract_nstations](#) for extracting the number of weather stations used at each time point to create the grids.

Examples

```
# The example shows how to build the netCDF data cubes.
# For extra example see \url{https://github.com/peterson-tim-j/BOMcatchr/blob/master/README.md}
#-----

# Set dates for building netCDFs and extracting data from 15 to 5 days ago.
startDate = as.Date(Sys.Date()-15,"%Y-%m-%d")
endDate = as.Date(Sys.Date()-5,"%Y-%m-%d")

# Set names for the netCDF file (in the system temp. directory).
ncdfFilename = tempfile(fileext='.nc')

# Build netCDF grids for daily precipitation and only over the defined time period.
dataErr_df = grid_build(ncdfFilename=ncdfFilename,
                        updateFrom=startDate,
                        updateTo=endDate,
                        vars = c('precip'))

# Now, to demonstrate updating the netCDF grids to one day ago, rerun with
# the same file names but \code{updateFrom=NA}.
dataErr_df = grid_build(ncdfFilename=ncdfFilename,
                        updateFrom=NA)

# Remove temp. file
unlink(ncdfFilename)
```

grid_sources

Source data URLs and attributes.

Description

grid_sources get available variables, units and URLs to BoM gridded data.

Usage

```
grid_sources()
```

Details

This function returns a list of available variables, unit, time step and URLs used to download the meteorological data.

Value

data.frame of the source data location and properties required by the package:

- `label`: string description of the variable.
- `units`: string for units of the variable.
- `time.step`: string for the time step of the data. days or months are accepted.
- `date.start`: string for the first date of the source data.
- `data.url`: string of URL to the source gridded data.
- `data.file.extension`: string for the file extension of the downloaded compressed source data.
- `data.file.format`: string for file extension to the file required within the downloaded file.
- `ncdf.name`: string for the name of the variable once input to the package netCDF file.
- `ellipsoid.crs`: string for Coordinate Reference System (CRS) for the gridded data ellipsoid.
- `update.days`: string for the update schedule of the source data from <https://www.bom.gov.au/climate/austmaps/update-schedule.shtml>. Each listed item in the string is the number of days after which the BoM update a data grid. That is, when the data for a given date is, say, 7 days old then the BoM may update the data for that day. This field is used to identify netCDF time points requiring update to the latest data.

Examples

```
vars = grid_sources()
```

<code>grid_summary</code>	<i>Summarise existing netCDF file.</i>
---------------------------	--

Description

`grid_summary` summarises the variables, units and date ranges of an existing netCDF file.

Usage

```
grid_summary(ncfile)
```

Arguments

`ncfile` file name of the netCDF data file built by this package.

Details

This function opens an existing netCDF file built using the package and returns a data.frame of variables, unit, stand and end dates of the data.

Value

data.frame summarising the attributes of each variable in the netCDF file. The data.frame includes the following columns:

- `group`: string for the netCDF group in which the variable is placed.
- `var.string`: string for the group and variable name.
- `from`: Date variable for the start of the first time step containing data.
- `to`: Date variable for the end of the last time step containing data.
- `time.step`: string for the time step of the data.
- `time.datum`: string for time datum from which netCDF layers are indexed.
- `units`: string for units of the variable.
- `ellipsoid.crs`: string for Coordinate Reference System (CRS) for the gridded data ellipsoid.

<code>weather_gauge_loc</code>	<i>Example weather gauge coordinates.</i>
--------------------------------	---

Description

Coordinates of the six example weather gauge within Australia. All six gauges have been selected to be approximately at the centre of a grid cell to minimise error introduced by interpolation of the extracted grid data.

Usage

```
data("weather_gauge_loc")
```

Format

data.frame of 3 columns and 6 rows.

Source

Bureau of Meteorology, Australia. See <https://www.bom.gov.au/climate/data/index.shtml>

weather_gauge_obs	<i>Example weather gauge observed data.</i>
-------------------	---

Description

Daily rain and maximum temperature observations (1/1/2010 - 31/12/2010) from six weather gauge within Australia. The data is used by a vignette to evaluate the extracted point data. All six gauges have been selected to be approximately at the centre of a grid cell to minimise error introduced by interpolation of the extracted grid data.

Usage

```
data("weather_gauge_obs")
```

Format

data.frame of 4 columns and 365*6 rows.

Source

Bureau of Meteorology, Australia. See <https://www.bom.gov.au/climate/data/index.shtml>

Index

* datasets

weather_gauge_loc, [16](#)

weather_gauge_obs, [17](#)

approx, [4](#)

ET.Abtew, [4](#)

ET.HargreavesSamani, [4](#)

ET.JensenHaise, [4](#)

ET.Makkink, [4](#)

ET.McGuinnessBordne, [4](#)

ET.MortonCRAE, [4](#)

ET.MortonCRWE, [4](#)

ET.Turc, [4](#)

extract_data, [2](#), [9](#), [14](#)

extract_layer, [7](#)

extract_locations, [3](#), [6](#), [8](#)

extract_nstations, [6](#), [10](#), [14](#)

grid_ages, [6](#), [11](#), [14](#)

grid_build, [6](#), [8](#), [10](#), [11](#), [12](#)

grid_sources, [10–12](#), [14](#)

grid_summary, [14](#), [15](#)

ReadInputs, [5](#)

splinefun, [4](#)

weather_gauge_loc, [16](#)

weather_gauge_obs, [17](#)